DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT - IV : Schema Refinement (Normalization)

Easy handwritten notes with diagrams for students

Topics in this unit

1. Purpose of normalization and anomalies
2. Functional dependency, Armstrong's axioms, closure
3. Lossless-join and dependency-preserving decomposition
4. INF, 2NF, 3NF and surrogate key
5. Boyce-Codd Normal Form (BCNF)
6. Multivalued dependency and 4NF
7. Join dependency and 5NF

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Schema Refinement (Normalization): Purpose of Normalization or schema refinement, concept of functional dependency, normal forms based on functional dependency Lossless join and dependency preserving decomposition, (INF, 2NF and 3 NF), concept of surrogate key, Boyce-Codd normal form (BCNF), MVD, Fourth normal form (4NF), Fifth Normal Form (5NF).

Part 1 : Purpose of Normalization

1.1 What is Normalization?

Definition: **Normalization** (schema refinement) is the process of organising the tables of a database to **reduce redundancy** and avoid **anomalies** by breaking a large table into smaller, well-structured tables linked by keys. It uses **functional dependencies** and a series of **normal forms**.

Bad design example - all facts in ONE table:

Emp_id	Emp_name	Dept_id	Dept_name	Dept_head
E1	Ravi	D1	CSE	Dr. Rao
E2	Sita	D1	CSE	Dr. Rao
E3	Arun	D2	ECE	Dr. Iyer
E4	Meena	D1	CSE	Dr. Rao

Department data (D1, CSE, Dr. Rao) is repeated for every employee. This **redundancy** causes three kinds of anomalies:

Anomaly	Problem	Example above
Insertion anomaly	Cannot insert some data without other data	Cannot add a new department D3 until an employee joins it
Deletion anomaly	Deleting one fact loses another fact	Deleting Arun (E3) also deletes the only record of ECE and Dr. Iyer
Update (modification) anomaly	Same fact stored many times; update must change all copies	Changing CSE's head needs 3 row updates; missing one gives inconsistency

Goals of normalization

- Remove **redundant data** and save space.
- Remove **insert / update / delete anomalies**.
- Keep data **consistent** and make the design easier to maintain.
- Decompose tables without losing information (lossless) and, if possible, without losing dependencies.

* **Trade-off:** Too much normalization gives many tables and more joins (slower reads). Sometimes designers do **denormalization** on purpose for speed.

Part 2 : Functional Dependency (FD)

2.1 Concept

Definition: A **functional dependency** X -> Y (read "X determines Y") means: if two tuples have the same value for X, they must have the same value for Y. X is the **determinant** (left side), Y is the **dependent** (right side).

Examples: **Roll_no -> Name**, **Dept_id -> Dept_name**, **(Roll_no, Course_id) -> Grade**. But **Name -> Roll_no** is not valid, because two students can have the same name.

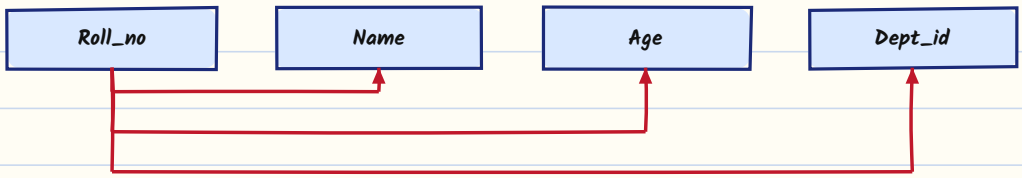


Fig 4.1 : FD diagram - Roll_no determines every other attribute (so Roll_no is a key)

Types of functional dependency

Type	Meaning	Example
Trivial	Y is a subset of X (always true)	{Roll_no, Name} -> Name
Non-trivial	Y is not a subset of X	Roll_no -> Name
Completely non-trivial	X and Y have no common attribute	Roll_no -> Name
Full dependency	Y depends on the whole of X (no proper subset of X is enough)	(Roll_no, Course_id) -> Grade
Partial dependency	Y depends on only PART of a composite key	(Roll_no, Course_id) -> Name, because Roll_no -> Name
Transitive dependency	X -> Y and Y -> Z (Y not a key) so X -> Z indirectly	Emp_id -> Dept_id -> Dept_name

2.2 Armstrong's Axioms (inference rules)

Rule	Statement
Reflexivity	If Y is a subset of X, then X -> Y
Augmentation	If X -> Y, then XZ -> YZ for any Z
Transitivity	If X -> Y and Y -> Z, then X -> Z
Union (derived)	If X -> Y and X -> Z, then X -> YZ
Decomposition (derived)	If X -> YZ, then X -> Y and X -> Z
Pseudo-transitivity (derived)	If X -> Y and WY -> Z, then WX -> Z

Armstrong's axioms are **sound** (they derive only valid FDs) and **complete** (they derive all valid FDs).

2.3 Closure of a Set of Attributes (X^+)

The closure X^+ is the set of all attributes that X determines. Algorithm: start with result = X; repeatedly add Y for every FD A -> Y where A is already inside result; stop when nothing new is added.

Example

R(A, B, C, D, E) with F = { A -> B, B -> C, CD -> E }

Step	Result	Reason
Start	{A, D}	X = AD
A -> B	{A, D, B}	A is in result, add B
B -> C	{A, D, B, C}	B is in result, add C
CD -> E	{A, B, C, D, E}	C and D in result, add E

So $(AD)^+$ = all attributes of R, hence **AD is a candidate key**. $A^+ = \{A,B,C\}$ and $D^+ = \{D\}$, so no smaller subset works.

Uses of closure

- Find **candidate keys**: X is a super key if X^+ contains all attributes of R.
- Test whether an FD X -> Y holds: check if Y is inside X^+.
- Check equivalence / cover of FD sets (canonical or minimal cover: single attribute on right, no extra left attribute, no redundant FD).

> * **Tip to find keys fast:** Attributes that never appear on the **right side** of any FD must be part of every key. Start with them and compute the closure.

Part 3 : Decomposition

Decomposition replaces a relation R by two or more smaller relations R1, R2, ... such that together they hold the same information. A good decomposition must be **lossless** and preferably **dependency preserving**.

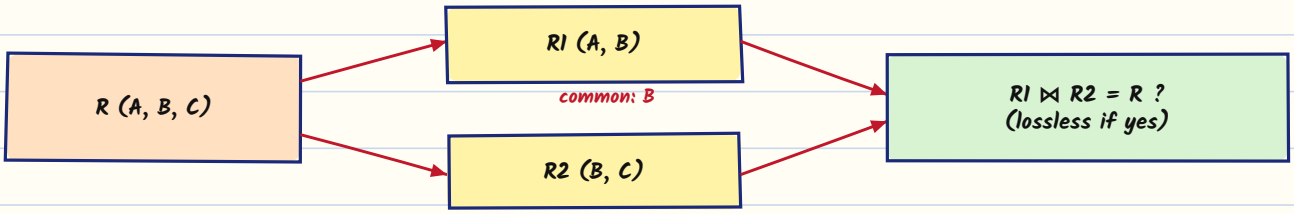


Fig 4.2 : Decomposition and re-joining

3.1 Lossless-Join Decomposition

After natural join of the parts we must get back **exactly the original relation** - no missing and no extra (spurious) tuples.

> **Definition:** Decomposition of R into R1 and R2 is **lossless** if and only if at least one of these holds in F^+: (R1 ∩ R2) -> R1 or (R1 ∩ R2) -> R2. That is, the **common attribute(s) must be a key** of at least one of the two parts.

Example: R(Emp_id, Dept_id, Dept_name) with Dept_id -> Dept_name. Split into R1(Emp_id, Dept_id) and R2(Dept_id, Dept_name). Common = Dept_id, and Dept_id -> Dept_name (key of R2) so it is **lossless**.

Bad split: R1(Emp_id, Dept_name) and R2(Dept_id, Dept_name) - common attribute Dept_name is a key of neither, so joining gives **spurious tuples** (lossy).

3.2 Dependency-Preserving Decomposition

Every FD of the original relation should be checkable inside a single decomposed table, without joining. Formally $(F1 \cup F2 \cup ...)^+ = F^+$.

Property	Why needed	Must always hold?
Lossless join	No information is lost or invented	**Yes - mandatory**
Dependency preservation	Constraints can be checked cheaply, without joins	Desirable (BCNF may not give it)

Part 4 : Normal Forms based on Functional Dependencies



Fig 4.3 : Each higher normal form includes (is stricter than) the previous one

Normal form	Removes	Condition (short)
1NF	Repeating groups / multi-valued attributes	All values atomic
2NF	Partial dependency	1NF + no non-key attribute depends on part of a key
3NF	Transitive dependency	2NF + no non-key attribute depends on another non-key attribute
BCNF	Anomalies from FDs whose determinant is not a key	For every FD X -> Y, X is a super key
4NF	Non-trivial multivalued dependencies	BCNF + for every MVD X ->> Y, X is a super key
5NF	Join dependencies not implied by keys	4NF + every join dependency is implied by candidate keys

4.1 First Normal Form (1NF)

A relation is in **1NF** if every attribute holds only **atomic (single, indivisible) values** - no multi-valued attributes, no repeating groups, no nested tables.

Not in 1NF:

Roll_no	Name	Phone
101	Ravi	98480, 98490
102	Sita	99000

Converted to 1NF (one value per cell; key becomes (Roll_no, Phone)):

Roll_no	Name	Phone
101	Ravi	98480
101	Ravi	98490
102	Sita	99000

Better: keep STUDENT(Roll_no, Name) and a separate PHONE(Roll_no, Phone) table.

4.2 Second Normal Form (2NF)

A relation is in **2NF** if it is in 1NF and has **no partial dependency**: every non-key attribute depends on the **whole** primary key. (A table with a single-attribute key is automatically in 2NF.)

ENROLL(Roll_no, Course_id, Student_name, Course_title, Grade) with key (Roll_no, Course_id)

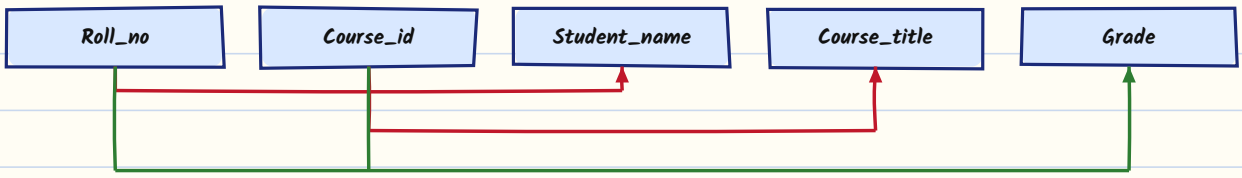


Fig 4.4 : Red = partial dependencies (bad); green = full dependency (good)

Decompose to 2NF:

- STUDENT(Roll_no, Student_name)
- COURSE(Course_id, Course_title)
- ENROLL(Roll_no, Course_id, Grade)

4.3 Third Normal Form (3NF)

A relation is in **3NF** if it is in 2NF and has **no transitive dependency**. Formally, for every non-trivial FD X -> A, either X is a **super key** or A is a **prime attribute** (part of some candidate key).

EMP(Emp_id, Emp_name, Dept_id, Dept_name) with Emp_id -> Dept_id and Dept_id -> Dept_name

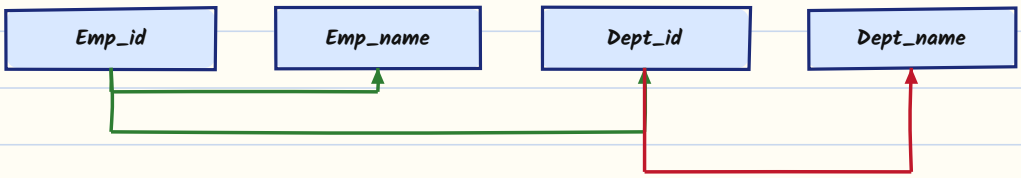


Fig 4.5 : Dept_id -> Dept_name (red) is a transitive dependency: Emp_id -> Dept_id -> Dept_name

Decompose to 3NF: EMP(Emp_id, Emp_name, Dept_id) and DEPT(Dept_id, Dept_name).

> * **3NF is always possible:** Every relation can be decomposed into 3NF with a **lossless join and dependency preservation** (3NF synthesis algorithm).

4.4 Surrogate Key

> **Definition:** A **surrogate key** is an artificial, system-generated identifier (usually an integer such as AUTO_INCREMENT / SERIAL / IDENTITY / sequence) with **no business meaning**, used as the primary key instead of a natural key.

	Natural key	Surrogate key
Source	Real-world data (Email, Aadhaar, ISBN)	Generated by DBMS (1, 2, 3 ...)
Meaning	Has meaning for the business	No meaning
Changes	May change (email, name change)	Never changes
Size	Often long or composite	Short, single integer; fast joins
Example	Roll_no = '23A05402'	Student_sk = 1007

- Use when no good natural key exists, the natural key is composite / long / changeable.
- Disadvantages: extra column, user cannot recognise rows; still keep a UNIQUE constraint on the natural key.

- Example: **CREATE TABLE STUDENT (Student_sk INT AUTO_INCREMENT PRIMARY KEY, Roll_no VARCHAR(10) UNIQUE, Name VARCHAR(40));**

4.5 Boyce-Codd Normal Form (BCNF)

> **Definition:** A relation is in **BCNF** if for **every non-trivial FD X -> Y, X is a super key**. BCNF is slightly stronger than 3NF (3NF allows Y prime; BCNF does not).

Example: TEACH(Student, Subject, Teacher) with FDs: {Student, Subject} -> Teacher; Teacher -> Subject (each teacher teaches only one subject).

- Candidate keys: (Student, Subject) and (Student, Teacher).
- Teacher -> Subject: Teacher is **not a super key** -> violates BCNF. But Subject is a prime attribute, so the table is in 3NF.

Decompose to BCNF: R1(Teacher, Subject) and R2(Student, Teacher). The join is lossless (common attribute Teacher is a key of R1), but the FD {Student, Subject} -> Teacher is **no longer preserved**.

Point	3NF	BCNF
Condition	X is super key OR Y is prime	X must be a super key
Redundancy	Small redundancy may remain	Removes all FD-based redundancy
Lossless join	Always possible	Always possible
Dependency preservation	Always possible	Not always possible

> **Exam tip:** Every BCNF relation is in 3NF, but not every 3NF relation is in BCNF. Order: 1NF ⊃ 2NF ⊃ 3NF ⊃ BCNF ⊃ 4NF ⊃ 5NF (each is a subset of the previous).

Part 5 : Multivalued Dependency and 4NF

5.1 Multivalued Dependency (MVD)

> **Definition:** An MVD X ->> Y (X multi-determines Y) holds when each value of X is associated with a **set of values of Y**, and this set is **independent** of the other attributes of the relation. It arises when a table holds two or more **independent multi-valued facts** about the same entity.

COURSE_INFO(Course, Teacher, Book): a course can have several teachers and several books, independently.

Course	Teacher	Book
DBMS	Rao	Korth
DBMS	Rao	Navathe
DBMS	Iyer	Korth
DBMS	Iyer	Navathe

Here Course ->> Teacher and Course ->> Book. Every teacher is combined with every book, causing redundancy even though the table is in BCNF (only trivial FDs, key = all three attributes).

- Every FD is also an MVD (X -> Y implies X ->> Y).
- MVD rules: complementation (X ->> Y implies X ->> R-X-Y), augmentation, transitivity.
- Trivial MVD: Y is a subset of X, or X ∪ Y = all attributes of R.

5.2 Fourth Normal Form (4NF)

A relation is in **4NF** if it is in BCNF and has **no non-trivial multivalued dependency** unless the determinant is a super key. Decompose by separating the independent multi-valued facts:

COURSE_TEACHER	
Course	Teacher
DBMS	Rao
DBMS	Iyer

COURSE_BOOK	
Course	Book
DBMS	Korth
DBMS	Navathe

4 rows become 2 + 2 rows; adding a new teacher now needs only **one** row instead of one per book.

Part 6 : Join Dependency and SNF

6.1 Join Dependency (JD)

A relation R satisfies a **join dependency** *(R1, R2, ..., Rn) if R can be split into R1 ... Rn and rebuilt exactly by their natural join. An MVD is a special JD with two parts (n = 2).

6.2 Fifth Normal Form (SNF / PJNF)

> **Definition:** A relation is in **SNF** (Project-Join Normal Form) if it is in 4NF and every join dependency in it is implied by its **candidate keys**. In simple words: the table cannot be split into smaller tables **without information loss** unless the split is based on keys.

Example: SPJ(Supplier, Part, Project) - a supplier supplies a part to a project only if (supplier supplies part) AND (supplier works for project) AND (project uses part). The rule is a 3-way constraint, so the table can be split into three binary tables:

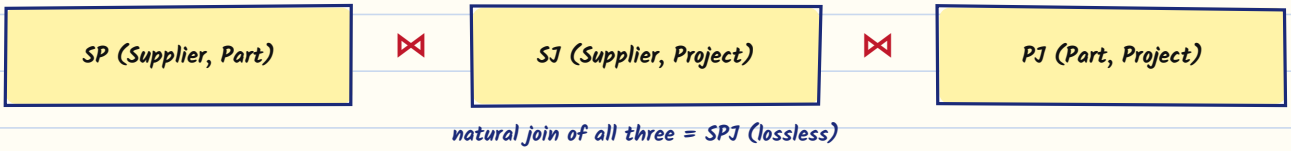


Fig 4.6 : SNF decomposition of SPJ into three tables

Normal form	Dependency it deals with	Typical problem solved
1NF	Atomic values	Multi-valued cells
2NF	Partial FD	Dependence on part of key
3NF	Transitive FD	Non-key determines non-key
BCNF	FD with non-key determinant	Remaining FD anomalies
4NF	Multivalued dependency	Independent multi-valued facts
5NF	Join dependency	Rare 3-way constraints

Quick Revision - Unit IV

- Normalization removes redundancy and insert / update / delete anomalies.
- FD X -> Y: same X means same Y. Types: trivial, non-trivial, full, partial, transitive.
- Armstrong's axioms: reflexivity, augmentation, transitivity. Closure X+ finds keys and checks FDs.
- Lossless join: common attribute must be a key of R1 or R2. Dependency preserving: (F1 ∪ F2)+ = F+.
- 1NF: atomic values. 2NF: no partial dependency. 3NF: no transitive dependency.
- BCNF: for every FD X -> Y, X is a super key (3NF may allow Y prime). BCNF may lose dependency preservation.
- Surrogate key: system-generated, meaningless, never-changing primary key.
- MVD X ->> Y: independent multi-valued facts. 4NF: BCNF + no non-trivial MVD.
- 5NF: no join dependency other than those implied by candidate keys.

Possible exam questions

1. What is normalization? Explain insertion, deletion and update anomalies with an example.
2. Define functional dependency. Explain Armstrong's axioms and attribute closure with an example.
3. Explain lossless-join and dependency-preserving decomposition.
4. Explain 1NF, 2NF and 3NF with examples.
5. Define BCNF. Differentiate 3NF and BCNF with an example.
6. What is a surrogate key? Compare with a natural key.
7. Explain multivalued dependency and 4NF with an example.
8. Explain join dependency and 5NF with an example.